

M1.1: Dynamic Array & Amortized Analysis

Theory: Contiguous memory-mapped arrays supporting $O(1)$ random access. Expands by doubling capacity and reallocating all items.

Details: Array expansion costs $O(N)$ but occurs infrequently, yielding an amortized $O(1)$ insertion cost. Continuous allocation allows hardware cacheline prefetching, maximizing L1/L2 hits.

Trade-off: Reallocations trigger tail latencies. If memory capacity bounds are known, initialize arrays with exact capacity to avoid runtime copies.

M1.2: Linked List Reference Overhead

Theory: Non-contiguous data nodes linked by memory references. Supports $O(1)$ structural insertions/deletions at known pointer locations.

Details: Lacks random access (requires $O(N)$ index walks). Pointer storage in 64-bit platforms leads to memory bloat-ing. Random node allocations trigger frequent cache misses.

Trade-off: Not suitable for read-heavy operations. If elements are small, pointers consume more space than data. Use static arrays or unrolled lists for performance.

M1.3: Stack & Queue Memory Alignments

Theory: Restricted linear structures enforcing LIFO (Stack) and FIFO (Queue) execution disciplines.

Details: Arrays provide efficient stack backings. Queues are best implemented as circular buffers to avoid element shifts. Double-linked list backings offer clean pointer mutations but add allocation overheads.

Trade-off: Circular arrays (Ring Buffers) are optimal for lock-free CAS queues. Linked queues cause memory fragmentation under high allocations.

M1.4: Hash Table Collisions & Load Factors

Theory: Maps keys to bucket indexes via hash functions for $O(1)$ average read/write times.

Details: Resolution via Chaining (linked lists or balanced trees) or Open Addressing (linear/quadratic probing, double hashing). Load factor $\alpha = N/M$ controls collision rates.

Trade-off: As α approaches 1, lookup degrades to $O(N)$. Java 8 addresses chaining degradation by promoting long lists to Red-Black trees.

M1.5: Incremental Rehash Execution

Theory: Resizes hash table slots when thresholds are crossed, redistributing existing key mappings.

Details: Direct synchronous rehashing blocks execution. Redis employs incremental rehashing, utilizing 'ht[0]' and 'ht[1]' concurrently, distributing migration workloads across incremental mutations.

Trade-off: Incremental rehashing requires double memory allocations and checks both tables, slightly increasing read latencies.

M2.1: BST Degradation & Balance Demands

Theory: Binary Search Tree (BST) property: left sub-tree values $<$ root $<$ right sub-tree values. Supports sorted in-order traversal.

Details: Ideal BST operations take $O(\log N)$ time. Monotonic input values (sorted insertions) degrade trees into single-linked linear lists, dropping performance to $O(N)$.

Trade-off: Balanced structures must be maintained. Self-balancing strategies like AVL or Red-Black trees rotate node subtrees dynamically during writes.

M2.2: AVL vs Red-Black Tree Rotations

Theory: AVL trees enforce strict height balancing (difference ≤ 1); Red-Black trees enforce weak balance using 5 node coloring rules.

Details: AVL yields faster lookups due to strict balance but requires high rotation costs. Red-Black trees require at most 3 rotations for any insertion or deletion, optimizing write throughput.

Trade-off: Red-Black trees are preferred in memory managers (e.g., Linux VMA) and collections (e.g., 'TreeMap') for balanced write-heavy performance.

M2.3: Heap Storage & Priority Queues

Theory: Max/Min heap is a complete binary tree where parent nodes are always larger/smaller than their children. Key structure for Priority Queues.

Details: Stored compactly in one-dimensional arrays without pointer overhead: left child is at $2i+1$, right is at $2i+2$, parent is at $\lfloor (i-1)/2 \rfloor$. Mutations take $O(\log N)$ via SiftUp and SiftDown.

Trade-off: Random searches cost $O(N)$. To support fast index updates, integrate a hash map tracking key positions within the heap array.

M2.4: Trie Node Memory & Radix Tree Compression

Theory: Prefix tree matching character segments of keys, sharing common prefixes to accelerate string retrievals.

Details: Lookup takes $O(L)$ where L is key length. Standard tries populate nodes with fixed pointer slots, yielding sparse allocations and high empty pointer overheads.

Trade-off: Radix trees fold linear single-child paths into single compound nodes, resolving memory expansion while preserving prefix lookups.

M2.5: Graph Matrix vs Adjacency List

Theory: Represents nodes and connections. Modeled via dense Adjacency Matrices or sparse Adjacency Lists.

Details: Adjacency Matrix checks connections in $O(1)$ but takes $O(V^2)$ space. Adjacency List consumes $O(V + E)$ space but checks connections in list scan times.

Trade-off: Adjacency lists are suited for sparse graphs. Dense graphs or GPU matrix algorithms perform better with matrices.

M3.1: BFS & DFS Graph Traversals

Theory: Systematically visits all graph nodes. DFS explores branches deeply; BFS visits neighbors level-by-level.

Details: DFS uses recursion or stacks (memory scales with depth); BFS uses queues (memory scales with width). BFS is optimal for finding shortest paths in unweighted graphs.

Trade-off: Tracking visited nodes via set flags is mandatory to avoid infinite loops when cycles exist.

M3.4: Binary Search Midpoint Overflow

Theory: Divide-and-conquer search on sorted arrays, narrowing search ranges by half in $O(\log N)$ time.

Details: Standard midpoint formula $\text{'(low + high) / 2'}$ overflows for large index values. Secure calculations require $\text{'low + (high - low) / 2'}$ or unsigned bit shifts.

Trade-off: Strict boundary checks (using 'left <= right' or 'left < right') are required. Requires static, pre-sorted datasets to be effective.

M3.5: Bitwise Operations & Two's Complement

Theory: Manipulates binary fields directly using CPU register bitwise logic (AND, OR, XOR, NOT, shifts).

Details: Signed integers use Two's Complement, letting ALU adders process subtraction directly. XOR is self-reversing. The mask $\text{'n \& (n - 1)'}$ drops the lowest set bit.

Trade-off: Complex bitwise expressions decrease readability. Be careful: signed shifts ('>') retain the sign bit, while unsigned shifts ('>>') fill empty slots with 0.

M4.3: Greedy Local Optimality

Theory: Selects local optimal choices at each stage, aiming for a global optimal solution.

Details: Highly efficient since it avoids backtracking. To be valid, problems must satisfy the greedy choice property and optimal substructure.

Trade-off: Greedy choices do not guarantee global optimality for all problems (e.g., 0-1 Knapsack). Mathematical proofs of correctness are required before use.

M3.2: Merge Sort vs Quick Sort

Theory: High-efficiency sorting strategies. Merge Sort is stable with a constant $O(N \log N)$; Quick Sort is unstable with an average $O(N \log N)$ but worst-case $O(N^2)$.

Details: Merge Sort requires $O(N)$ temporary memory. Quick Sort runs in-place (stack depth $O(\log N)$). Poor pivot choices degrade Quick Sort recursion depths to $O(N)$.

Trade-off: Modern libraries use IntroSort: running Quick Sort, switching to Heap Sort if stack depths exceed limits, and inserting on small subsets.

M3.3: Heap Sort Cache Performance

Theory: In-place unstable sorting using a binary heap, extracting elements and sorting them via SiftDown operations in $O(N \log N)$ time.

Details: Runs in-place with zero external allocations. However, heap adjustments read distant array indexes, breaking cacheline prefetching and causing high cache misses.

Trade-off: Despite its stable $O(N \log N)$ bounds and zero space overhead, heap sort is slower than Quick Sort or Merge Sort on real CPUs due to cache miss latency.

M4.1: Recursion & System Call Stacks

Theory: Self-referencing functions using the runtime system stack to evaluate sub-problems.

Details: Every call creates a Stack Frame storing registers, parameters, and return addresses. Deep recursion without base cases throws 'StackOverflowError' crashes.

Trade-off: Tail recursion optimizations (where supported) fold stacks. For deep recursions, convert code to iterative loops backed by user-space collections.

M4.2: DP Memoization vs Tabulation

Theory: Evaluates overlapping subproblems. Solves them once and caches results.

Details: Top-down with Memoization tables (recursive) or Bottom-up with Tabulation tables (iterative). Rolling arrays compress state tables from $O(N)$ to $O(1)$.

Trade-off: Requires subproblems to be independent and free of side effects. Designing optimal state transition formulas is complex.

M4.4: Divide & Conquer & Master Theorem

Theory: Breaks problems into independent sub-problems of similar types, solving them recursively and combining their results.

Details: Complexity of recurrence relations $T(N) = aT(N/b) + f(N)$ is solved via the Master Theorem, classifying bounds into three asymptotic cases.

Trade-off: If sub-problems share overlapping dependencies, divide-and-conquer degrades to exponential execution times. Use dynamic programming instead.

M5.1: CPU Cacheline & Spatial Locality

Theory: Microprocessors use multi-level cache hierarchies (L1/L2/L3) to speed up memory access, transferring data in 64-byte Cache Lines.

Details: Spatial locality benefits from contiguous array allocations. Concurrent updates to independent variables on the same cache line trigger False Sharing, causing MESI bus contention.

Trade-off: Align hot variables to separate cache line boundaries using buffer padding or '@Contended' flags to avoid execution stalls.

M5.2: Virtual Memory & TLB Page Walks

Theory: Abstracts memory access via page maps. MMUs translate virtual addresses to physical pages.

Details: Translation tables reside in main memory. To avoid page walk latency, MMUs cache translations in a Translation Lookaside Buffer (TLB). TLB misses cause page table walks.

Trade-off: Random memory hops trigger TLB misses. For massive heap applications, configure Huge Pages to expand TLB ranges and reduce page walk depths.

M5.3: Process vs Thread Context Switch

Theory: Task schedulers swap execution contexts on CPU cores.

Details: Process switches require swapping virtual memory mappings by writing to register CR3, which flushes TLB caches and incurs high latency. Thread switches share page tables and only swap registers.

Trade-off: Excessive thread creation causes scheduling bottlenecks. Restrict thread counts using pools, or implement user-space fibers/coroutines.

M6.1: Design Patterns Core

Theory: Standard solutions to common design challenges, classified into Creational, Structural, and Behavioral patterns.

Details: Promotes decoupling. Singletons manage global states; Proxies intercept calls; Decorators extend stream filters; Observers handle event emissions.

Trade-off: Introducing abstract layers increases class counts. Avoid applying design patterns to simple, stable systems where direct structures are sufficient.

M6.4: Scale-Out System Foundations

Theory: Architecting high-concurrency systems using horizontal scaling (Scale-Out) instead of upgrading single servers (Scale-Up).

Details: Leverages stateless service nodes, load balancers (L4/L7), and database sharding to distribute read/write traffic.

Trade-off: Distributed systems face latency and consistency challenges, requiring tradeoffs defined by the CAP theorem.

M5.4: TCP State Machine & Network Troubleshooting

Theory: Establishes connections via three-way handshakes and terminates them using four-way handshakes.

Details: The active-closing endpoint enters 'TIME_WAIT' for 2 MSL to ensure the final ACK arrives and to clear delayed packets from the network.

Trade-off: High-concurrency short connections accumulate thousands of 'TIME_WAIT' slots, exhausting ports. Enable 'tcp_tw_reuse' or use persistent connections.

M6.2: Test-Driven Development & Mocking

Theory: Development discipline focusing on the Red-Green-Refactor loop.

Details: Write a failing test first, implement minimum code to pass, and refactor. Mock objects isolate test scopes by simulating database or network calls.

Trade-off: Writing test code increases upfront time, but it reduces debugging overheads later. Mock configurations must be updated when external APIs change.

Zone T1: C.S. Low-Level APIs

```
BinarySearch(arr, key), QuickSort(arr, low, high),
Heapify(arr, n, i), BFS(graph, start), DFS(graph, start),
RadixTree.compress(), TLB_flush()
```

Zone T2: System Execution Faults

```
stack_overflow_recursion, out_of_memory_heap,
hash_table_collision, tcp_port_exhaustion,
cpu_cache_false_sharing, tlb_miss_penalty, integer_overflow_mid
```

M5.5: Asymptotic Bounds & Master Theorem Limits

Theory: Classifies runtime growth rates using Big-O (upper bound), Big-Omega (lower bound), and Big-Theta (tight bound).

Details: Ignores coefficients and focuses on dominant terms. The Master Theorem resolves recurrence equations for recursive branchings.

Trade-off: Big-O reflects $N \rightarrow \infty$ trends. For small datasets (e.g., $N < 100$), the low-level overhead of recursion or memory lookups makes $O(N^2)$ insertion sort faster than $O(N \log N)$ quicksort.

M6.3: Clean Code & SOLID Principles

Theory: Clean code practices to maintain code readability and reduce technical debt. SOLID comprises five object-oriented design principles.

Details: SRP (Single Responsibility), OCP (Open-Closed), LSP (Liskov Substitution), ISP (Interface Segregation), and DIP (Dependency Inversion). Keep cyclomatic complexity under 10.

Trade-off: Refactoring must be backed by unit tests. Clean code separation may increase class counts, but it simplifies maintenance and extensions.